

소프트웨어 vs 하드웨어

- 소프트웨어 : 코드나 명령이 순차적으로 저장된 것

폰노이만 구조와 하버드 아키텍처

- 초기 컴퓨터 : 미리 정해진 로직에 따라 작동, 선을 쫓고 빠는 방식으로 로직 변경
- 폰노이만 구조 : 명령어를 메모리에 저장, CPU가 이를 읽고 해석; 메모리에 모든 명령어와 데이터를 저장
 - 장점 : 구조 단순, 비용 낮음
 - 단점 : 메모리 접근 시 병목현상. CPU와 메모리를 동시에 읽을 수 X. CPU와 메모리 사이 시스템 버스가 하나이기 때문
- 하버드 아키텍처 : 명령어와 데이터를 각각 다른 메모리에 저장, 시스템 버스가 분리됨
 - 장점 : 명령어 & 데이터를 동시에 읽음 -> 성능 우수
 - 단점 : 구조 복잡, 비용 높음
- 폰노이만 구조의 개선
 - 캐시메모리 : CPU/주기억 장치 간 캐시메모리; 인텔 L1/L2/L3 등 -> 병목 해결
 - 하버드 아키텍처와 병행 : 칩에 내장된 캐시메모리 명령어 캐시, 데이터 캐시로 나눔

CPU의 기능

- 데이터 처리 : 산술연산 및 논리 연산 수행. 사칙연산 및 논리연산 수행, ALU는 여기서 핵심 모듈
- 데이터 이동 : 메모리<->메모리, I/O <-> 메모리 로 데이터를 이동 가능
- 흐름제어 : 프로그램의 실행 흐름을 제어. 명령어들은 순차적으로 시행되나 조건문과 같은 제어문을 통해 실행순서 변경 가능

CPU의 구조

- 컨트롤 유닛 : 명령어(기계어)를 읽고 해석 후 ALU, 레지스터에게 명령
- 산술 논리 연산 장치(ALU) : 데이터의 연산을 수행. 산술연산 + 논리연산 수행. 결과값을 레지스터에 저장
- 레지스터 : CPU 내부의 고속 저장장치; 명령과 데이터를 임시저장.
 - 프로그램 카운터(PC) : 다음에 실행할 명령어의 주소 저장
 - 명령어 레지스터(IR) : 현재 실행중인 명령어를 저장
 - 메모리 주소 레지스터(MAR) : 메모리에 접근하기 위한 주소를 저장
 - 메모리 버퍼 레지스터(MBR) : 메모리에서 읽어온/쓸 데이터를 임시 저장
 - I/O address resister, I/O buffer resister

컴퓨터의 구조

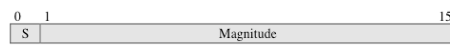
- 중앙처리장치(CPU) : 컴퓨터의 동작 제어, 데이터 처리
- 메모리 : 데이터(와 명령어)를 저장
- I/O : 입력모듈 + 출력모듈
- 시스템버스(시스템 상호 연결)
 - 주소버스 : 메모리 주소를 지정하는데 사용
 - 데이터버스 : 데이터를 전송하는데 사용
 - 제어버스 : 제어신호를 전송
- CPU: 명령어, 데이터 읽음 -> CU : 명령어 해석 -> ALU : 데이터 처리 -> I/O or 메모리로 전달

CPU 동작 과정 (가상 CPU)

- 16비트의 명령어; 앞의 4비트는 명령어인 OP코드로 구성. 나머지는 주소를 나타냄
 - OP코드 : 0001 -> load, 0010 -> store, 0101 -> add라고 하자
 - AC : CPU코어 내의 레지스터
- 1. Fetch : PC가 가리키는 메모리 주소에서 명령어를 읽어 IR에 저장하는 과정
 - PC가 가리키는 메모리 주소(ex) 300번지)에서 명령어가 읽혀 나옴
 - 읽힌 명령어는 CPU 내부의 IR에 저장(ex) 1940)
- 2. Decode : IR의 명령어를 CPU의 컨트롤 유닛이 해석, 가상 CPU에서는 OP코드를 분석해 어떤 연산을 수행하는지 결정 ex) 1 = LOAD
- 3. 실행 : CPU의 실행유닛의 ALU를 사용해 수행. 데이터와 관련된 연산인 경우 OP코드 외에 address를 사용해 메모리 주소를 지정/ 레지스터를 지정
 - 1940 : 940번지 메모리 데이터를 읽어 AC에 저장
- 4. 다음 Fetch 준비 : 다음 명령어를 준비하기 위해 PC값을 업데이트 ex) 300 -> 301



(a) Instruction format



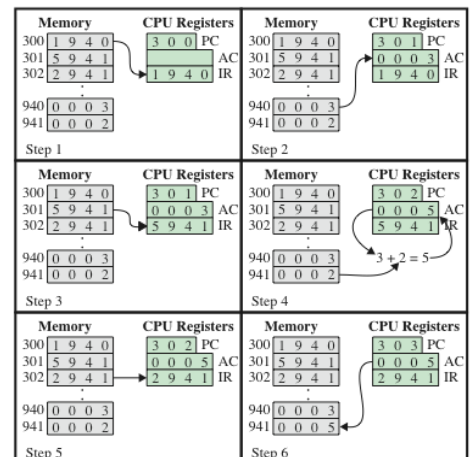
(b) Integer format

Program Counter (PC) = Address of instruction
Instruction Register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

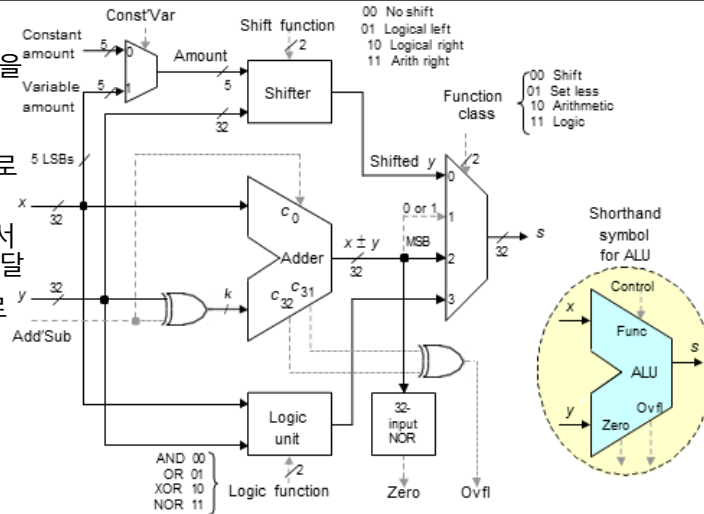
0001 = Load AC from Memory
0010 = Store AC to Memory
0101 = Add to AC from Memory

(d) Partial list of opcodes

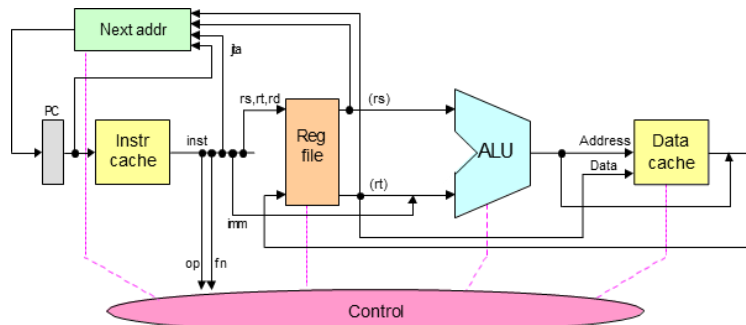


CPU 내부 구조

- ALU : 산술연산 + 논리연산을 수행; OP코드라는 신호에 의해 연산을 결정
 - 뺄셈은 2의 보수 방식으로 덧셈으로 처리
 - ALU의 출력은 멀티플렉서 (MUX)를 통해 외부로 전달
 - 곱셈/나눗셈 -> 시프터로 구현



- 레지스터 세트 : CPU 내부의 임시 저장. ALU가 필요한 데이터나 연산 결과 보관
 - 레지스터 파일 : 사용자가 직접 접근 가능;
 - 접근 불가 특수목적 레지스터 : PC, IR, MAR, MBR
 - 컴파일러 -> 변수들을 레지스터에 할당
- Control unit : IR에 저장된 명령 해석. 그 결과를 CPU 내부의 요소를 제어하는 제어신호 생성
- CPU 내부 버스 : address bus, data bus등으로 나뉨



Abstract view of the instruction execution unit for MicroMIPS.

- Next addr -> 다음 주소를 만드는 곳; PC값을 1 증가
- Cache -> 외부 메모리 내용을 임시로 저장
- Reg file -> 레지스터 파일 = 레지스터 세트

ISA (Instruction Set Architecture; 명령어 집합구조)

- 특정 CPU가 이해하고 실행할 수 있는 명령어의 집합 -> 컴퓨터 시스템 설계 첫단계
 - RISC : 명령어의 개수를 줄이고, 각 명령어를 빠르게 실행. Hardwired 방식 구현
 - CISC : 다양하고 복잡한 명령어. 마이크로프로그래밍 방식으로 구현

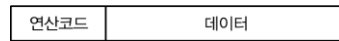
어드레싱 모드

- 명령어에서 피연산자(데이터)의 위치를 지정하는 방법

→ Immediate Mode :

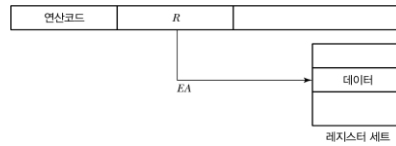
명령어 내에 실제 데이터 포함

-> ADD ACC, #1234



→ Register Mode : 명령어 내에 데이터가 저장된 레지스터 번호가 포함

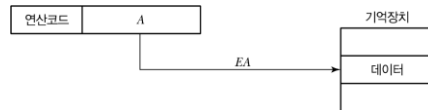
-> ADD R0, R1, R2



→ Direct Mode :

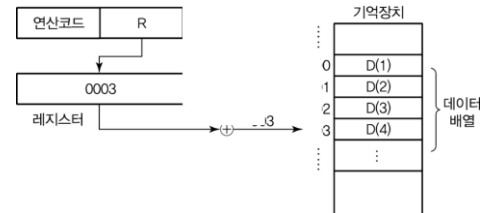
명령어 내에 데이터가 저장된 메모리의 실제 주소 포함

-> LOAD ACC, 900



→ Register Indirect Mode : 명령어에 메모리 주소가 저장된 레지스터의 번호가 포함

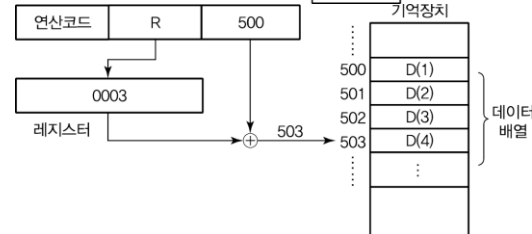
-> LOAD ACC, (R0)



→ Register Indirect Mode with Index : 레지스터에 저장된 주소값에 명령어에 포함된 오프셋값을 더해 실제 메모리 주소를 계산.

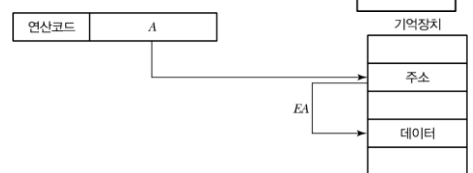
Register with Displacement라고도 함

-> LOAD ACC, 500(R3)



→ Memory Indirect Mode : 명령어의 메모리 주소에 실제 데이터의 주소가 저장

-> LOAD ACC, @900



→ Implicit Addressing mode : 명령어 수행에 필요한 데이터 위치가 미리 지정

Control Unit 구현 방식

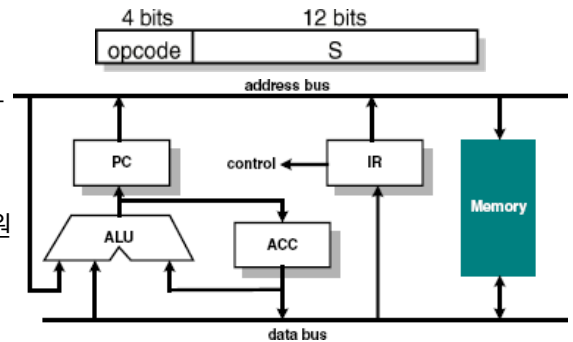
- Hardwired 방식 : 제어신호 생성 로직을 하드웨어 회로로 구현, 복잡하지만 속도 빠름. RISC에서 채용
- Microprogrammed 방식 : 제어 신호들을 마이크로프로그램으로 ROM에 저장. 이를 실행해 제어 신호 생성. 유연성이 높지만 속도 느림, CISC에서 채용

오퍼랜드 시스템

- 명령어 -> 연산의 대상이 되는 데이터(오퍼랜드)를 가질 수 있음
 - 오퍼랜드는 source와 destination으로 구분. 각각 원래 데이터가 있던 곳, 연산 결과가 저장될 곳임.
 - 3-operand system : 명령어 하나에 2개의 소스 오퍼랜드와 1개의 목적지 오퍼랜드
 - 2-operand sys -> 2 소스 오퍼, 1 dest 오퍼
 - 1-operand sys -> 명령어 하나에 한 오퍼랜드(주로 소스), 목적지는 ACC와 같이 묵시적으로 지정

CPU 내부 데이터 흐름

- 다음과 같은 아키텍처의 MU0 CPU가 있다고 가정
 - MU0는 그림과 같은 ISA를 제공
 - 또한 메모리 다이렉트 모드만 지원
- 1) Fetch : PC에서 메모리로 명령어가 담긴 주소를 찾아 메모리에서 IR로 명령어를 로드
 - PC -> add bus -> mem -> data bus -> IR
- 2) Instruction decode : IR에서 OP코드를 분석. control unit에선 명령어에 맞는 신호 생성. PC값은 add bus를 통해 K값 혹은 1만큼 증가
 - IR -> CU -> ALU/MEM/ACC ...
 - PC -> ALU -> PC
- 3) Operand Fetch : IR에서 피연산자의 주소를 add bus로 메모리에 전달. 피연산자의 주소가 data bus를 통해 ALU등에 로드
 - IR -> MEM -> data bus -> ALU -> (ACC)
- 4) Execute Instruction : ALU에서 실제로 연산을 수행. 그리고 그 결과를 ACC에 저장
 - ACC -> ALU -> ACC
- 5) write back : IR에서 메모리로 저장할 주소를 전달, ACC에서 연산한 결과를 메모리에 저장
 - IR -> MEM, ACC -> MEM



Instruction	Opcode	Semantics
LDA S	0000	$ACC := mem_{re}[S]$
STO S	0001	$mem_{re}[S] := ACC$
ADD S	0010	$ACC := ACC + mem_{re}[S]$
SUB S	0011	$ACC := ACC - mem_{re}[S]$
JMP S	0100	$PC := S$
JGE S	0101	if $ACC \geq 0$ $PC := S$
JNE S	0110	if $ACC \neq 0$ $PC := S$
STP	0111	stop

단일 사이클의 성능

- Ins cache -> 2ns, ALU OP -> 2ns, Res read -> 1ns, Res write -> 1ns
- ADD r3, r2, r1 -> 6ns, LOAD r5, r6 -> 8ns, STOR (r6), r5 -> 7ns;
 - 각 명령어의 종류에 따라 이것이 성능에 미치는 영향을 가중평균 가능
 - 암달의 법칙 : 시스템 일부를 향상 -> 전체 시스템 성능?
 - $Speedup = 1 / ((1 - P) + (P / S))$ (P: 향상되는 부분의 비율, S: 그 부분의 성능 향상 배율)

CPU의 퍼포먼스

- CPU 바운드 : 전체 시스템 성능이 CPU에 성능 때문에 제한
- IO 바운드 : 전체 시스템 성능이 입출력 장치 성능 때문에 제한

CPU 속도의 기본적인 방법들

- 클럭 증가 -> 소재의 한계 /전압 감소 등의 문제
- 좋은 소재 사용 -> 반도체 소자 성능을 향상
- 회로 설계 개선 -> 많은 발전이 있어 개선 어려움

파이프라인

- -> '컴퓨터 구조'의 관점에서 속도 ↑
- 명령어 처리 과정을 여러 단계로 나눠 동시에 서로 다른 작업들이 처리; 병렬성 높임 (학생증 만드는 과정에서 학생인증-사진찍기-계산-발급을 기다리지 않고 여러 사람이 동시에)
- 파이프라인 4단계 예시 : Fetch - Decode - Execute- Write
 - 필요에 따라 더 세분화하거나 줄일 수 있음 : 5단계, 3단계 등
- 파이프라인 유형
 - Pipeline
 - Super pipeline : 가장 오래 걸리는 단계를 여러 단계로 쪼갬
 - Super scalar : 다중 병렬 처리 과정
 - -> Super pipelined super scalar

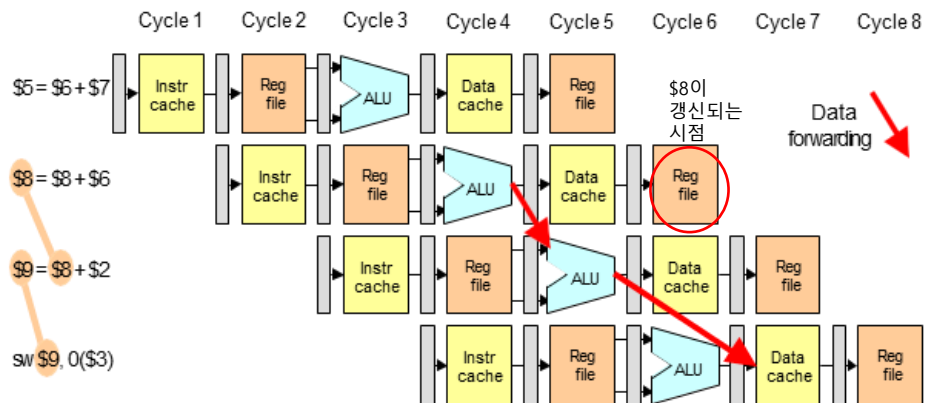


Single pipelining

- 병렬처리를 위해 몇 개의 동작을 명령어 수행 중 각 단계를 한번만 중첩
- 1클럭에 1Inst(1cycle)만 처리한다고 가정;
 - (FI-DI-EI-WB)에서 4개의 Inst를 실행하는데 필요한 시간? -> 7 clerk
 - 100개? -> 103clerk, 성능 향상 = $103/400 = \text{약 } 25\%$ 향상
 - 1000개? -> 1003clerk, 성능향상 = $1003/4000 = \text{약 } 25\%$ 향상
- 명령어에 따라 cycle 차이; but, 파이프라인에선 건너 뛴 수 없음 (반찬 안 먹고 싶어도 단계를 건너 뛴 수 없음)

파이프라인의 문제점

- structure hazard
 - FI, EI 단계가 동시에 기억장치를 사용할 경우 기억장치 충돌에 따른 지연시간
 - Data bus, address bus가 하나밖에 없어서
 - > address/data bus를 서로 분리 = 하버드 아키텍처

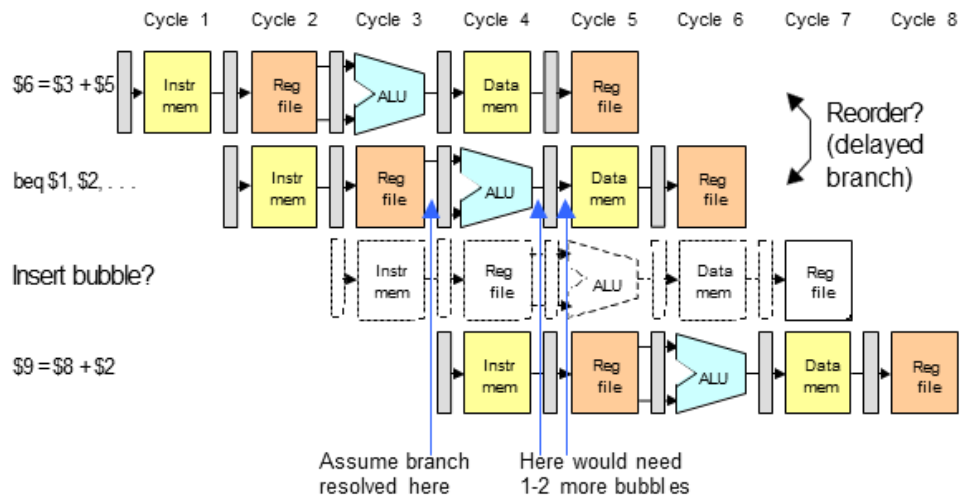


- Data hazard
 - Data forward :
 - 그림의 \$8 = \$8 + \$6 이후 \$9 = \$8 + \$2 에서 8번 레지스터를 불러올 때, 8번 레지스터는 아직 위의 명령어에 의해 갱신되기 이전의 시점
 - > Data forward를 통해 (위쪽 빨간 화살표) 레지스터에 쓰는 것과 병행해서 ALU에 데이터 공급
 - Instruction reordering :
 - 오른쪽 코드에서 $c = c + 1$ 을
 - 2번째와 3번째 줄 사이에 끼워 넣으면
 - data hazard 일어나지 않음; -> data forward 없이 해결
 - > 누가? 컴파일러가 해결
 - Data hazard 해결?
 - 1) software -> Instruction reordering
 - 2) hardware -> Data forward
 - 3) 아무 일도 안 하는 clerk (bubble) 추가
- $$\begin{aligned}
 a &= b + c \\
 c &= k + 1 \\
 d &= a + 3 \\
 c &= c + 1
 \end{aligned}$$
- (\$6) -> ALU의 결과가 아닌 Data mem
 #1 LDA \$5, (\$6)
 #2 ADD \$4, \$5, \$6
 #3 ADD \$4, \$3, \$2
 -> \$6과 관련 없는 명령어를 중간에 위치
 bubble 필요 X (#3을 #1, #2중간에 위치)
 의존성 -> 결과가 업데이트 되기 전
 다시 입력됨

Control dependency



Control Dependency

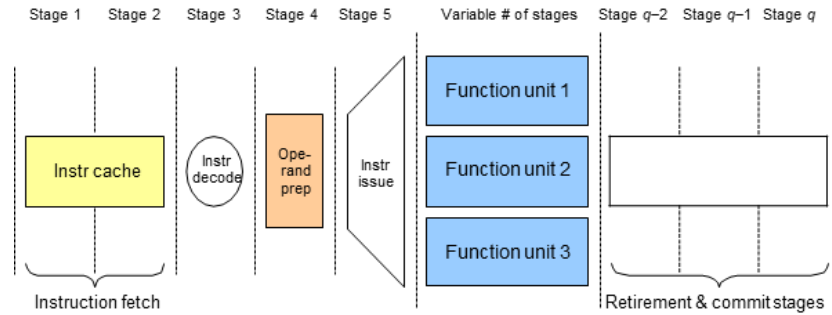


- 상황 :
프로그램 진행중 beq문을 만남
-> 분기 할지 말지 정해지지 않음
-> CPU는 다음 실행할 명령어를 load. 그런데 분기 여부에 따라 다음 명령어 주소 다를 수 있음
-> 만약 CPU가 잘못된 주소의 명령어를 가져오면 파이프 라인을 비우고 다시 명령어를 load 해야함(Plusing) = 성능 저하
- 문제점 : if문(beq)으로 분기가 달라지면 이전 분기꺼는 실행 X
- 해결 1? : 이전 거를 모두 지운다 (Plusing)
→ 해결 2 : 이전 명령어와 순서를 바꾼다 = delayed branch

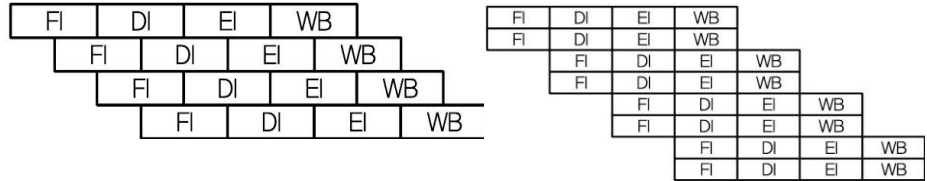
Branch Prediction

- Beq면 결과를 예측해서 fetch 분기를 바꿈
→ Taken이 높은 확률 :
for, while처럼 루프문 일때, 명령문이 위로 JMP면 루프라고 가정
→ 하드웨어 발전 : 둘 다 pre fetch하고 필요 없는 건 갖다 버림

파이프라이닝 심화 기법

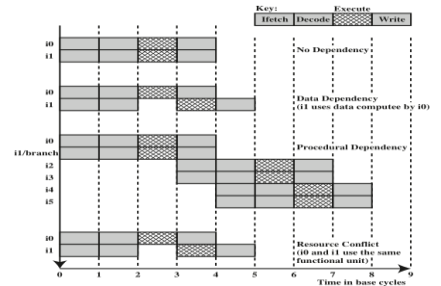


- Instr cache : 메모리 -> 매우 느림; 느리므로 쪼갬다 -> super pipelining
-> 가장 빠른 스텝에 클럭 속도를 맞춤
- Function unit 1,2,3 : 여러 개 동시에 execute -> super scalar; 하나로 할 때보다 단축
- $A = b * c \rightarrow d = e + 3 \rightarrow k++$; 곱셈하는 동안 여러 개의 명령 실행
-> 끝나는 순서대로 write? -> 결과가 달라질 수 있음
-> retirement & commit stages = write 할 때 어떤 순서대로 할 것인지 결정
- 슈퍼 파이프라이닝, 슈퍼스칼라



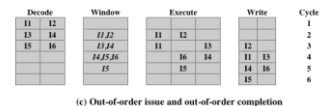
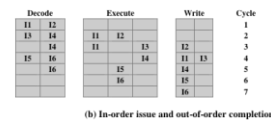
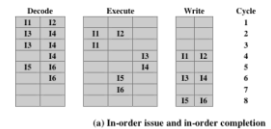
Dependency 효과 : 슈퍼스칼라

- Renaming:
 $K = a + 2$
 $a = h + c \rightarrow a_bar = h + c$
 $d = a + 3 \rightarrow d = a_bar + 3$
 → 같은 변수 a지만
 d에서의 a와 k에서의 a는 다르다
 -> $a' = b + c$; $d = a' + 3$ 으로 이름 변경



슈퍼스칼라의 명령어 이슈와 병합 정책

- In-order issue and in-order completion
-> 순서대로 와서 순서대로
- In-order issue and out-order completion
-> 순서대로 와서 역순으로
- Out-of-order issue and out-of-order completion
-> 순서 X -> 순서 X
→ Window : 어떻게 가장 빠른가?
-> 이걸 고려해 순서를 고려





컴퓨터 메모리 시스템의 주요 특징들

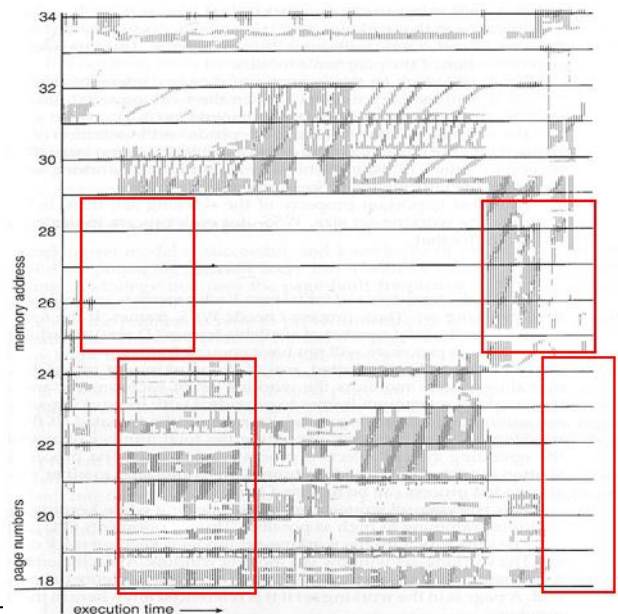
- 저장 위치 : 외부 vs 내부
- 용량 : 바이트 용량 vs 단어 수로 용량
- 단위 : 바이트 vs 단어 수
- 메모리 접근 방법 : sequential, Direct, random, associative
- 퍼포먼스(성능) :
 - Access time : 첫 data 읽을 때 까지의 시간
 - Cycle time: 개별 data 읽을 때 걸리는 시간
 - Transfer rate : 전송률
- Physical type(물리적 구성)
 - Semiconductor Magnetic Optical Magneto-optical
- Physical characteristics(물리적 특징)
 - Volatile/nonvolatile : 휘발성/비휘발성 : 전원끊김 -> 지워지나?
 - Erasable/nonerasable : 삭제가능/불가능

Memory hierarchy (아래로 내려갈수록 속도 ↓, 비용 ↓)

- Inboard memory (Internal memory)
 - Registers : ACC, IR, R0 ...
 - Cache : CPU 내부 메모리
 - Main memory
- Outboard memory (Outboard memory + Off-line storage = External memory)
 - Magnetic disk, CD-ROM, ..., BLU-RAY
- Off-line storage : Magnetic Tape -> 시퀀셜 메모리

Locality

- 메모리를 항상 다 쓰지 않음
 - = locality
 - 특정 지역만 사용
 - 전체가 빠를 필요 X
 - 적은 수의 메모리를 빨리 액세스

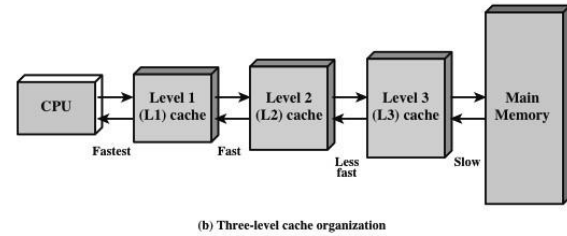
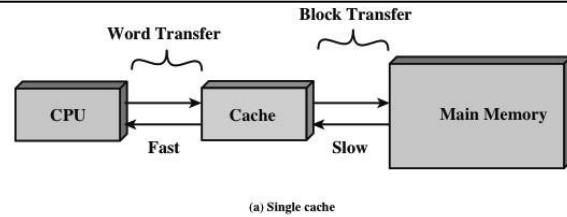


캐시와 메인 메모리

✓

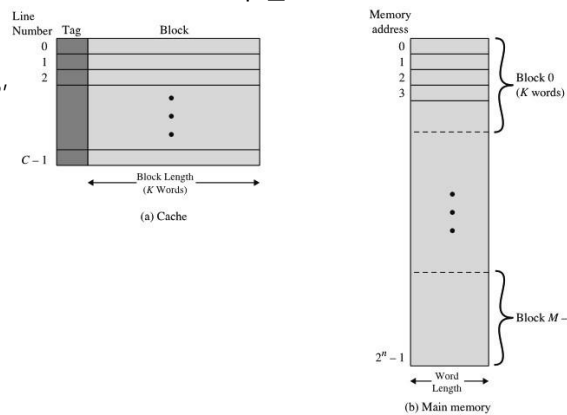
Cache and main memory

- CPU에서 데이터를 찾는 방법
 - 1) CPU -> 캐시로 접근
데이터가 있나? : 있으면 Hit
없으면 메모리로 -> miss
 - 2) miss면, 메모리 -> 캐시
- Single cache (위)
 - Cache : 필요한 만큼만 빠름
 - Main mem : 전부 빠를 필요 X
- 3-level cache (아래)
 - L1 : L1을 2개 사용하여 Inst/Data 구별 -> 하버드 아키텍처
 - L2, L3 : 폰노이만 아키텍처
 - L1, L2, L3 캐시로 하여금 modified havard architecture 구현



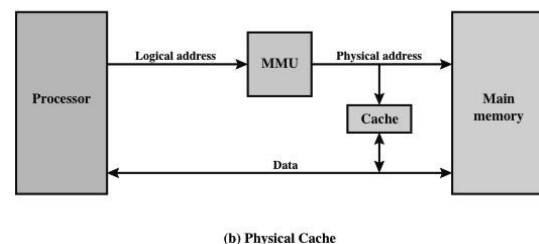
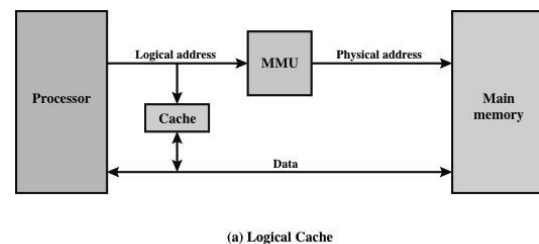
Cache/Main memory structure

- '어떻게 캐시를 쓰고 다시 비울 것인가?'
- Cache (왼)
 - Tag : 이걸 참조 -> '내가 원하는 data가 여기?'
- Main memory (오)
 - 각 블록에 대해 한 블록은 캐시 사이즈와 동일한 라인 넘버 가짐
- 작동방법 : CPU가 캐시 내에 data 찾을 -> hit면 ok, miss면 블록을 통째로 지우고 메인 메모리에서 블록 째로 가져와 다시 찾을 -> 비효율적



Logical and physical Cache

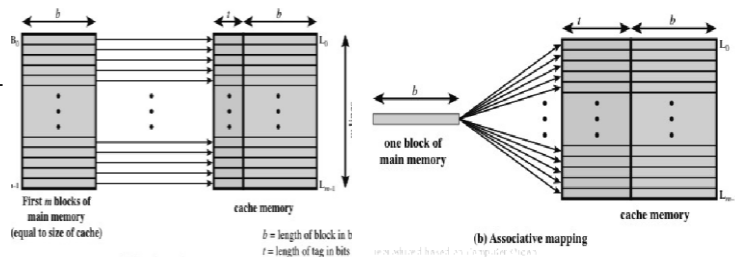
- Virtual memory : CPU는 mem이 없더라도 마치 있는 것 처럼 다룸
- Logical cache : logical address 에서 태깅
- Physical cache :
 - MMU : logi add <-> phy add 사이의 변환
 - Physical cache : phy add를 tagging
 - 모자란다면? Ex) LA=10TB, PA=10GB -> SSD/HDD 사용



Direct/Associative mapping

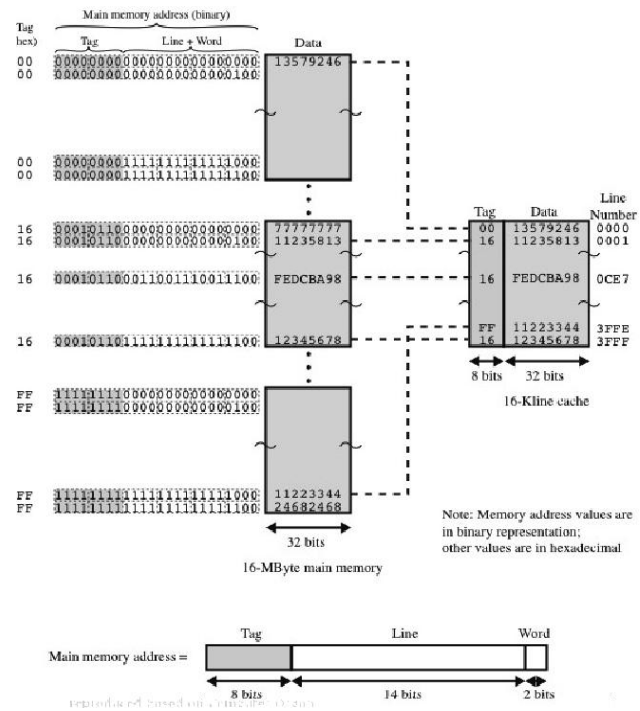
Direct mapping

- 메인 MEM
-> 캐쉬 MEM이 1:1 대응
→ 라인에 붙어있는 Tag가 몇 블록인지 보여줌



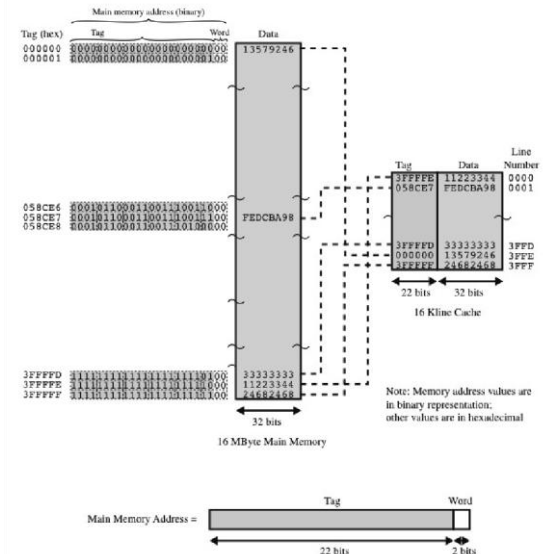
Direct mapping example

- 작동 원리 : 열람실에서 학번과 자리를 1:1 매칭
→ 학생은 학번 뒷자리 숫자에 배정된 좌석만 이용 가능
→ 다른 학생이 이용하고 싶으면 원래 학생 나가야 함
→ 학생을 찾는 방법: 학생이 학번 뒷자리와 매칭되므로 해당 좌석에 가서 확인만 하면 됨
- Tag : 몇번째 블록인지? (서로 다른 학번 앞자리)
- Line Number : 캐시 상에서의 주소 (서로 같은 학번 뒷자리)
- 문제점 : 같은 라인넘버 내에 계속 액세스
-> 계속 지웠다 써졌다 반복
-> 캐시가 있으나 마나함



Associative mapping : 문제의 해결법?

- 어느 학생이든 열람실 빈 자리에 아무 자리나 앉을 수 있음 (= 메모리의 어떤 블록이든 캐시의 어떤 라인으로 매핑 가능)
- 단점 : 데이터를 찾으려면 캐시 내의 라인을 일일이 찾아야 함



Set associative mapping/cache update

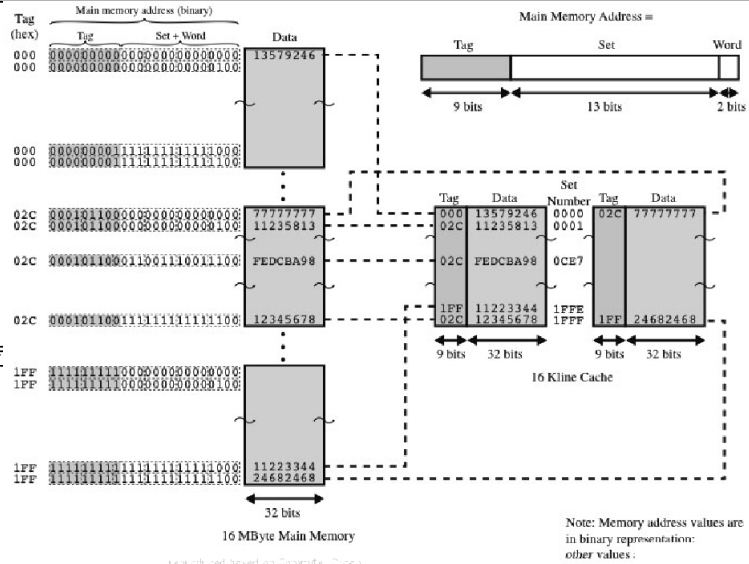


Set associative mapping

- Direct가 여러 세트
-> 여러 개의 열람실;
캐시의 세트 개수
-> 열람실의 수

2way set associative mapping example

- Virt. MEM
→ 캐시 -> Mem 포지션
→ 하드로 보낼 건데
어떻게?
- 캐시에서 데이터를
찾는 법
→ 두 캐시 세트의 Set
Number만 찾아내서 비교하면 됨 (지정좌석이 있으므로 열람실만 바뀌어야 하며 확인)
- 캐시 세트를 얼마나 늘려야 하나? -> 4way cache정도면 ok



Note: Memory address values are in binary representation: other values.

Replacement algorithms (캐시 업데이트 방식)

- Least recently used(LRU) : 가장 예전에 사용된 걸 교체
(가장 최근 = 또 사용될 가능성 높으므로; 로컬리티 -> 그 지역만 계속 씬)
- First in first out(FIFO) : 가장 일찍 들어온 걸 교체
- Least frequently used(LFU) : 가장 적게 사용된 걸 교체

Write policy

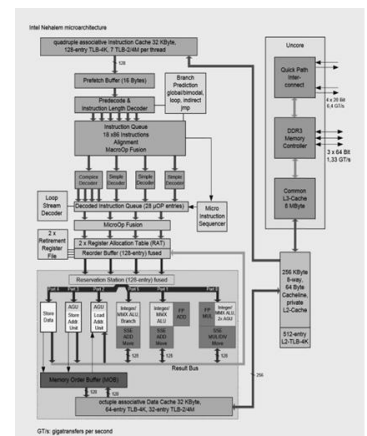
- '어떻게 쓸 것인가?' : 캐시에 쓰는 시간 필요
→ Write through : 읽은 다음에 쓰는 것 -> 성능 ↓, 심플
- Write back : 읽은 다음 사용하고 나중에 씬 -> 성능 ↑, 복잡
- WT & WB 문제? : 코어가 여러 개면 문제 발생

Multilevel cache

- 2-level cache : L1, L2 -> L2 적용시 hit ratio 증가

Intel Nehalem architecture (오)

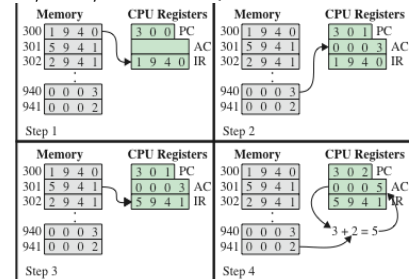
- 색이 있는 박스부분 : Inst.
- 색 박스 윗부분 : data
- L3 : 메모리로부터 읽음, 속도 빠르게함
- L2 : CPU 코어끼리 공유
- L1 : Inst/data를 분리





컴시 개요

- 폰노 : mem에 모든 inst -> 구조 단순, 비용 ↓; but, inst, data 동시?
-> 병목. 왜? Add bus, data bus 하나
- 하버드 : mem을 inst/data 분리
-> 병목 X; add bus, data bus 2개



컴퓨터 구조와 CPU 동작 과정

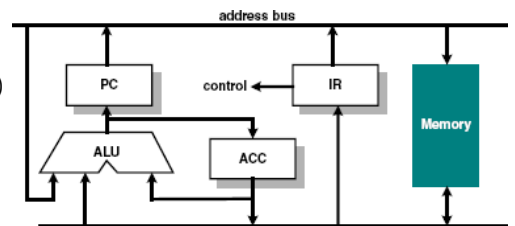
- COM = CPU(=CU+ALU+REG) + I/O + MEM
sys bus = add bus + data bus

CPU 내부구조, ISA

- ALU : (산술+논리)연산, (-) : 2의 보수, (×/÷) = 시프터
- 레지 세트 : 레지파일 = 사용자 접근 가능(acc, r0 ...), 접불 특목 레지 = (PC,IR,MAR,MBR) -> 컴파일러 : 레지에 변수 할당
- ISA : 특정 CPU 명령 집합
RISC : 명령 개수 ↓, 속도 ↑, HW 구현; CISC : 명령 개수 ↑, 속도 ↓, MP 구현
- Addressing mode
 - 포함 : immediate mode : ADD ACC #1234
 - 직접 : Register mode : ADD R0, R1
Direct mode : ADD R3, 900
 - 간접(참조) : Memory indirect mode : LOAD ACC (700)
Register indirect mode : LOAD ACC, (R5)
Register indirect mode with displacement : LOAD ACC 500(R2)
 - 암시 : implicit addressing mode : STP
- CU 구현 방식 : Hardwired : 물리적 회로, 속도 ↑, 복잡, RISC
Micro programmed : ROM에 제어 신호 저장, 속도 ↓, 유연, CISC

Data flow

- Operand system : operand(=피연산자)
= source + destination
n-operand sys : n개의 피연산자
- Data flow :
 - 1) Fetch : PC -> MEM -> IR
 - 2) Inst decode : IR -> CU -> PC/ACC/ALU/MEM ..., PC -> ALU -> PC
 - 3) Operand fetch : IR -> MEM -> ALU -> (ACC)
 - 4) Execute Inst : ACC -> ALU -> ACC
 - 5) Write back : IR -> MEM, ACC -> MEM



성능 향상과 파이프라인

- 암달 법칙 : 시스템 일부 향상 -> 전체 향상; $1/[(1-P)+(P/S)]$
- CPU(I/O) bound : 전체 성능이 CPU(I/O) 에 의해 제한
- 파이프라인 유형 : pipeline -> super scalar/pipeline -> super scalar pipeline

단일 파이프라인

- Structure hazard : FI, EI가 동시에 기억장치 -> 충돌, 지연 -> data bus/add bus 분리
- Data hazard : 결과값이 갱신되기 전에 입력값으로 호출
-> 해결 : data forward : 캐시에 쓰는 것과 동시에 필요한 곳에 data 공급
Inst reordering : 갱신 시점이 맞도록 의존성 문제 없는 코드를 재배치; bubble 삽입

Control dependency

- Beq(if)문 -> 분기 O? X? -> 분기 예측 실패시 fetch를 잘못 로드하면 파이프라인에서 명령어를 모두 삭제(flushing) -> 성능 ↓
- Branch prediction : beq면 결과 예측, fetch를 바꿈 -> inst가 위로 가면 : 루프문 간주
-> HW 발전 : 둘다 pre fetch, 필요X 버림

Advanced pipelining

- Super pipelining : 여러개 파이프 라인 -> 병렬처리
Super scalar : 가장 느린 스텝 -> 가장 빠른 스텝의 클럭에 맞도록 조깅
- Renaming : $K=a+2$, $a'=h+c$, $d=a'+3$
- 명령어 이슈, 병합정책 : in order-in order, in order-out order, out of order-out of order -> window는 가장 빠르게끔 배치

CPU 메모리 ↓ CPU 내부

- Inboard mem : Reg, cache, main mem
outboard mem : disk, CD ... BLU RAY
offline mem : magnetic tape
- Locality : 메모리를 다 쓰지 X, 특정 지역 사용 -> 적은 수의 메모리를 빠르게

캐시와 메인 메모리

- CPU에서 data? : (CPU -> 캐시) -> (miss? -> MEM)
- 3 level cache : L1*2 -> 하버, L2,L3 -> 폰노; L1,L2,L3 -> mod 하버
- Virt MEM : mem이 없어도 있는 것처럼; logi cache: logical addres에 태깅
physical cache : Phy add를 태깅; MMU = virt <-> logi; 모자라면? HDD/SSD

Direct/(Fully associative) mapping

- Direct mapping : 캐시-mem 1:1대응; tag=몇 블록?; line num=캐시상 주소;
문제 : 캐시상의 locality -> 성능 ↓
- Fully associative mapping : mem -> 캐시 간 자유 매핑; 단점 : 찾? -> 모든 라인 조사

Set associative mapping / cache update

- Set associative mapping : 여러 캐시set, 1블록 -> 1세트; 매핑 캐시 내에선 자유 매핑
data 찾는법 : set num을 찾음 -> 그 캐시 내에서 라인들을 조사
- Replacement algorithm : LRU, FIFO, LFU
- Write policy : write through, write back; 코어 ↑, 문제O
- Intel Nehalem : L3=메모리 읽음, 속도 ↑; L2=코어끼리 공유, L1=Inst/data 분리